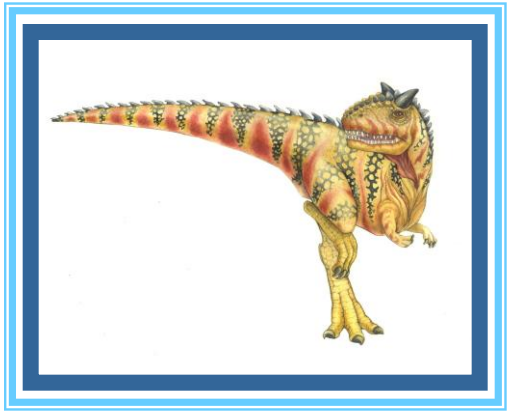
Chapter 3: Processes

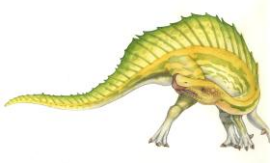


Chapter 3: Processes

- 3.1 Process Concept
- 3.2 Process Scheduling
- 3.3 Operations on Processes
- 3.4 Inter-process Communication (IPC)

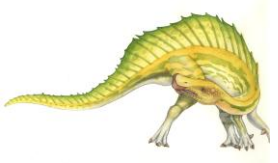
OBJECTIVES

- To introduce the notion of a process -- a program in execution, which forms the basis of all computation
- To describe the various features of processes, including scheduling, creation and termination, and communication
- To learn about interprocess communication



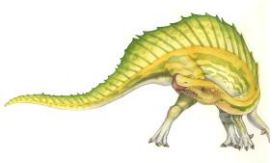
3.1 Process Concept

- **Process** – a program in execution; process execution must progress in sequential fashion
- Program is ***passive*** entity stored on disk (**executable file**), process is ***active***
  - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
  - Consider multiple users executing the same program
  - Example: If two users are running a word processor, two processes will be created.



3.1.1 Process Structure

- A process is more than the program code, which is sometimes known as the **text** section.
- It also includes the current activity:
  - The value of the **program counter**
  - The contents of the **processor's registers**.
- It also includes the process **stack**, which contains temporary data (such as function parameters, return addresses, and local variables)
- It also includes the **data section**, which contains global variables.
- It may also include a **heap**, which is memory that is dynamically allocated during process run time.



Process in Memory

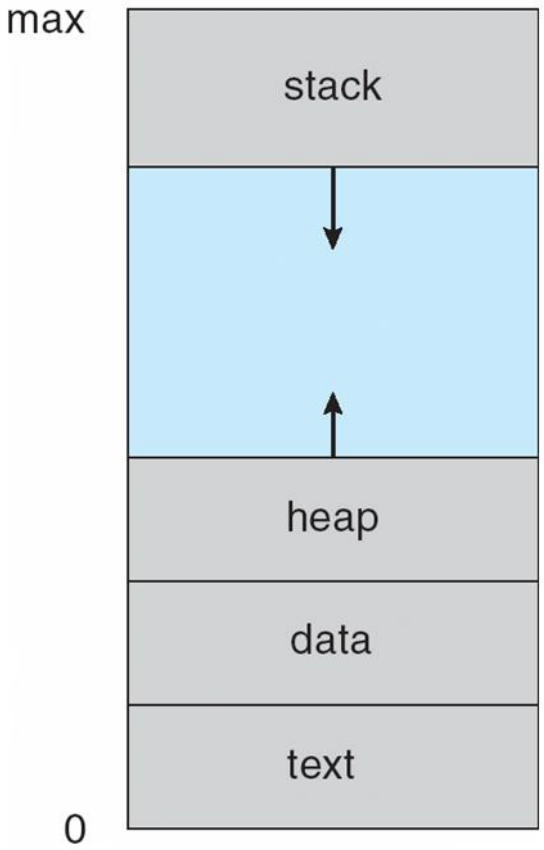
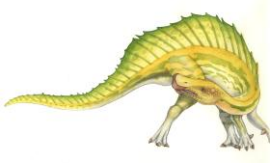


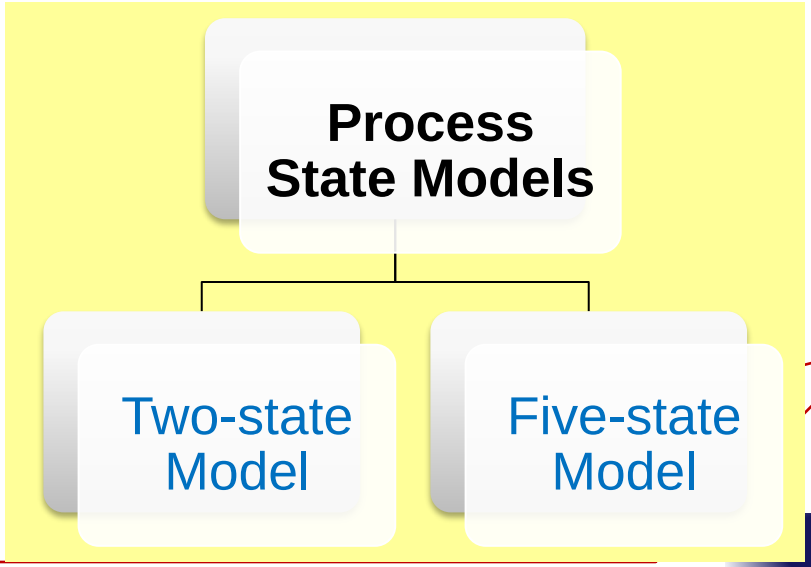
Figure 3.1



Process States

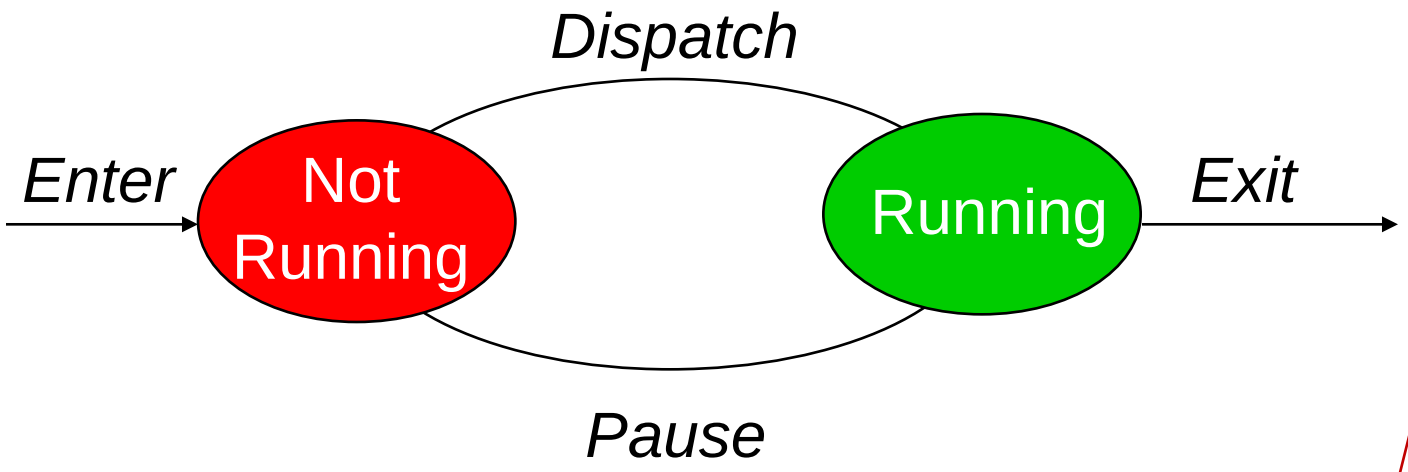
- Process states depict the **current activity** of a process.
- When a process is executed,
  - the process will change from one state to another;
  - changes states after an event occurs.

- A **state transition diagram** can tell what event causes a process to change states

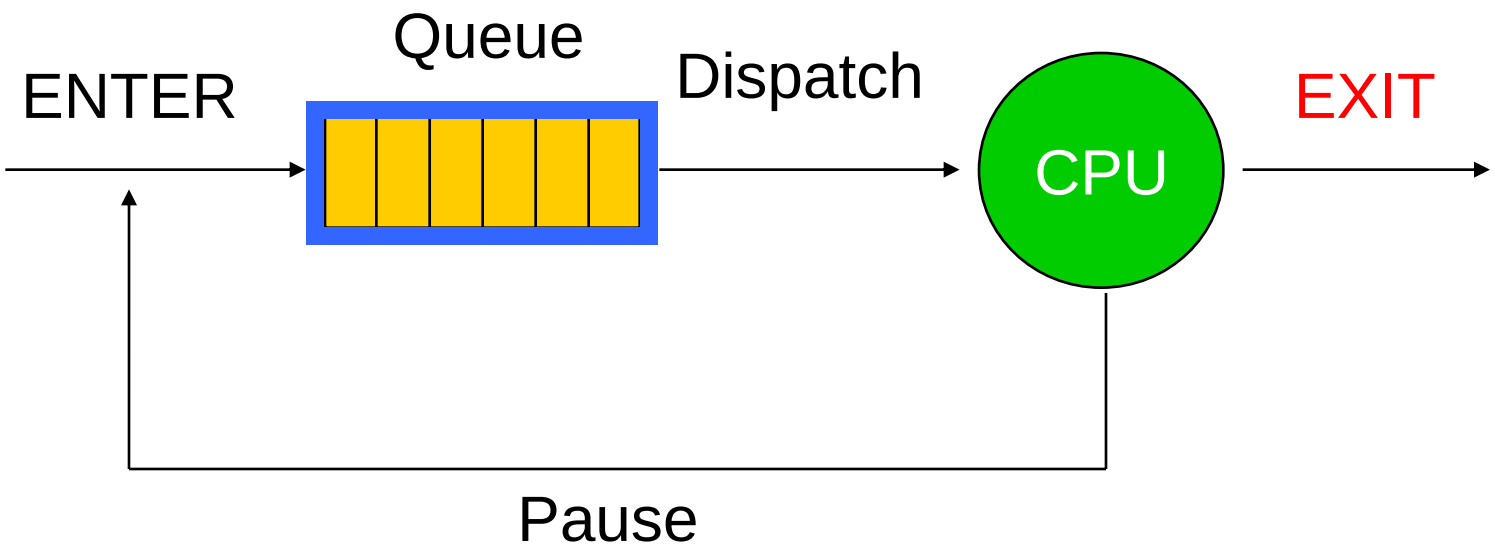


Process with Two-State Model

- A process can be in one of the following states:
  - Running
  - Not-running
- State Transition Diagram



Two-State Model : Queue Diagram



3.1.2 Process State (Five State Model)

- As a process executes, it changes **state**
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution
- State Transition Diagram:

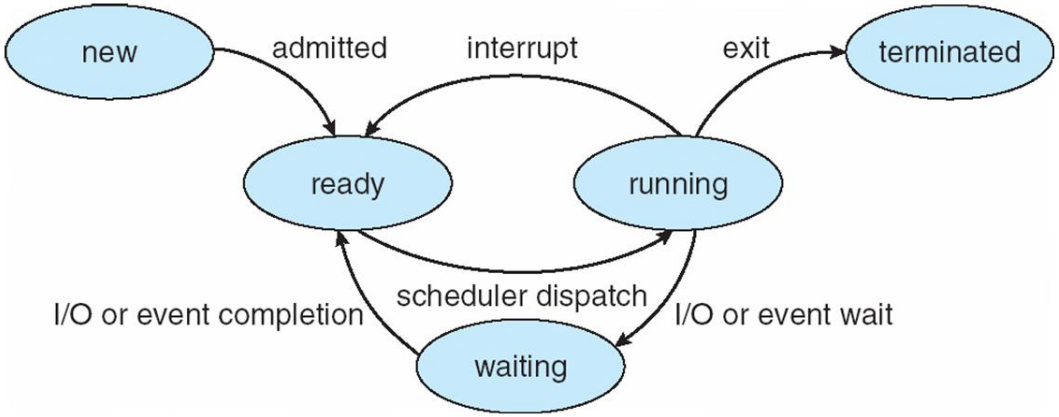
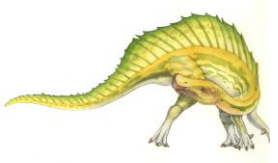


Figure 3.2 Diagram of process state



Five State Model: Queue Diagram

- **Queuing diagram** represents queues, resources, flows

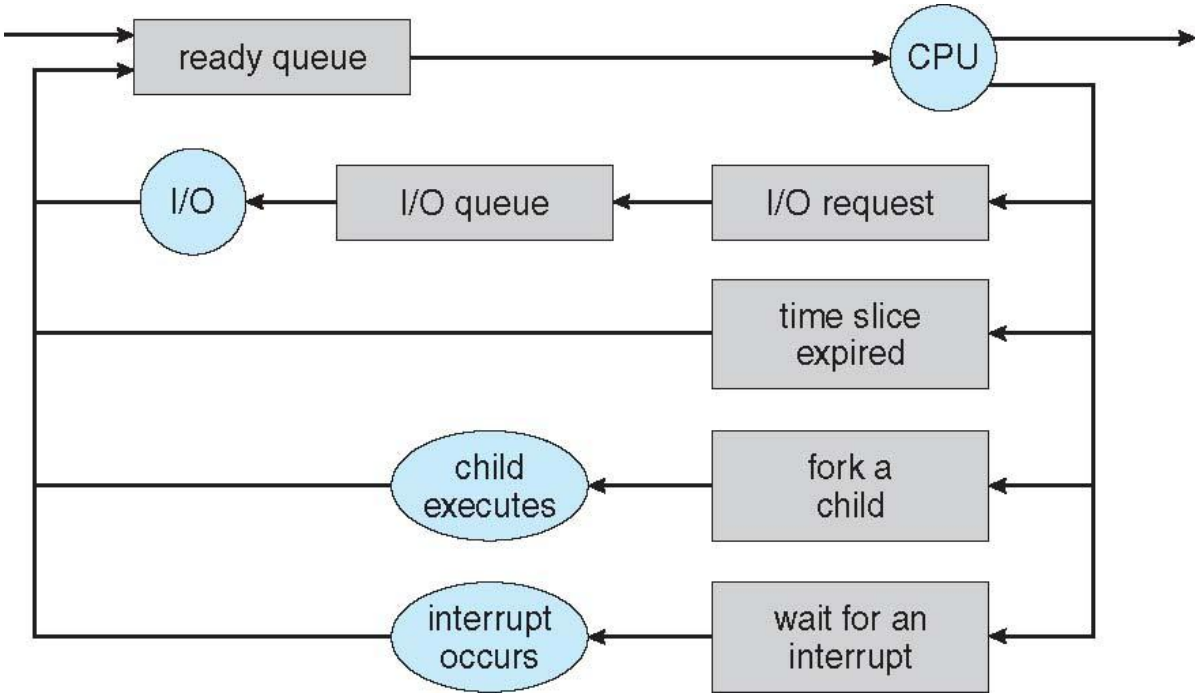
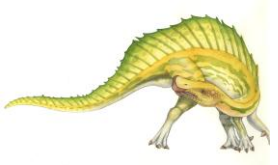
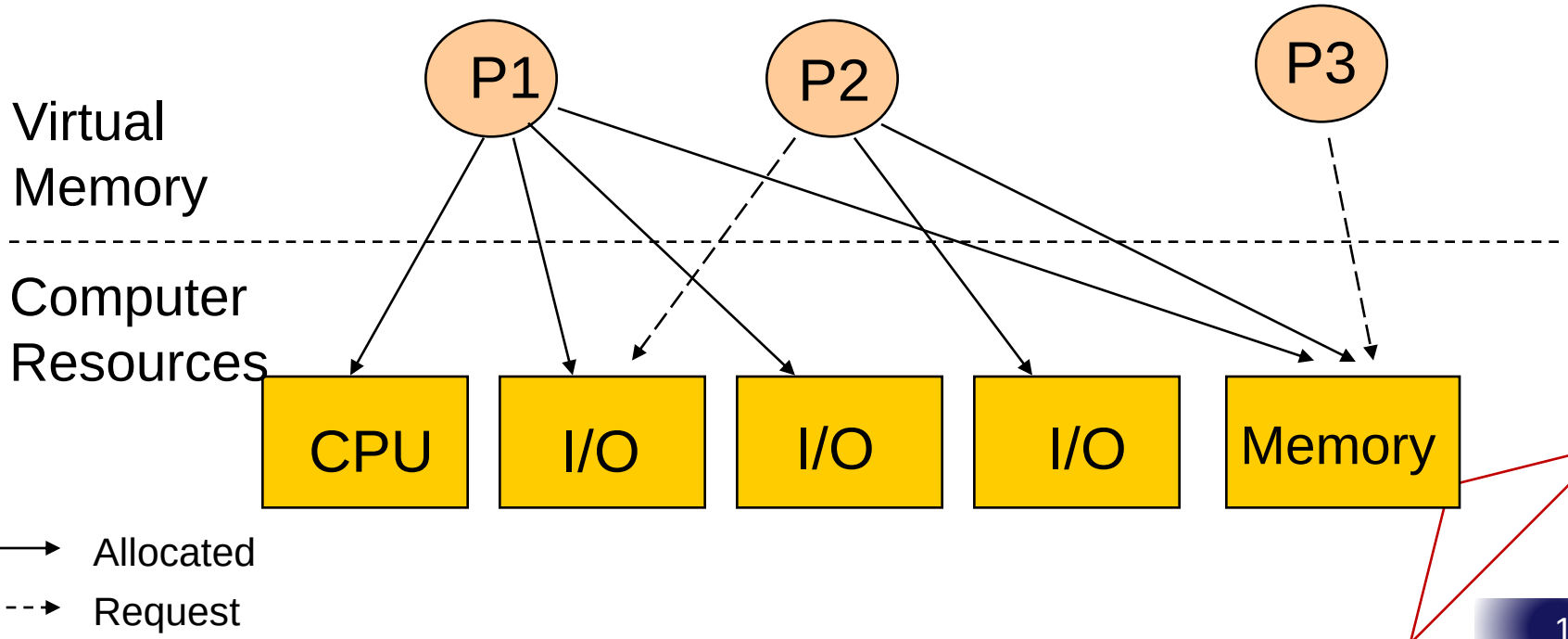


Figure 3.6 Queueing-diagram representation of process scheduling



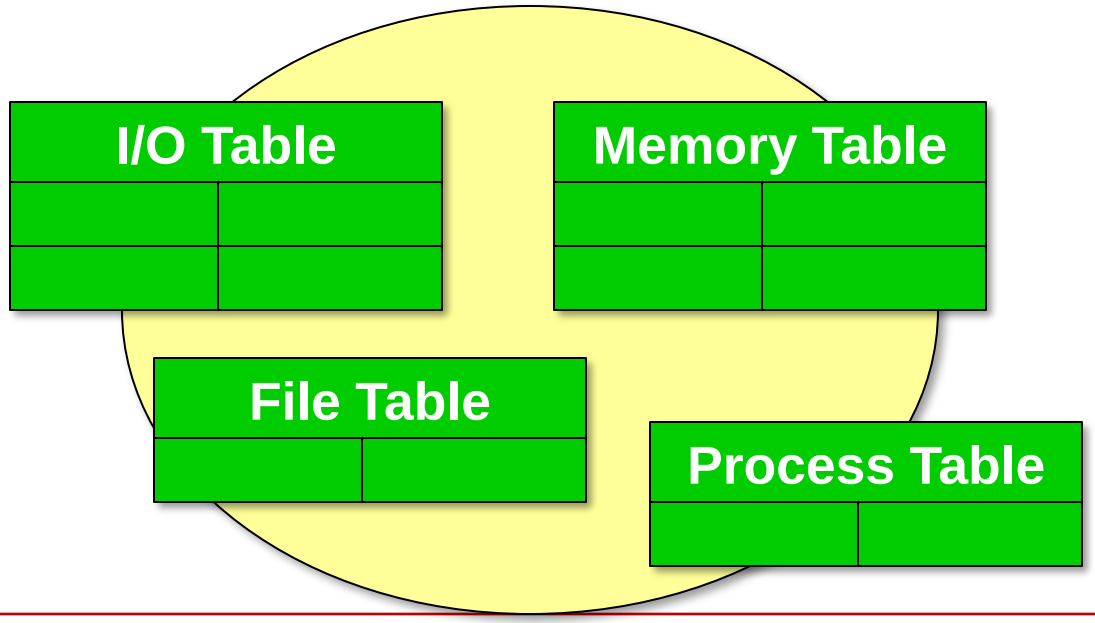
Process Description

- OS controls all the events that occur in a computer system.
- Therefore some data structure is needed to **keep track** of this information

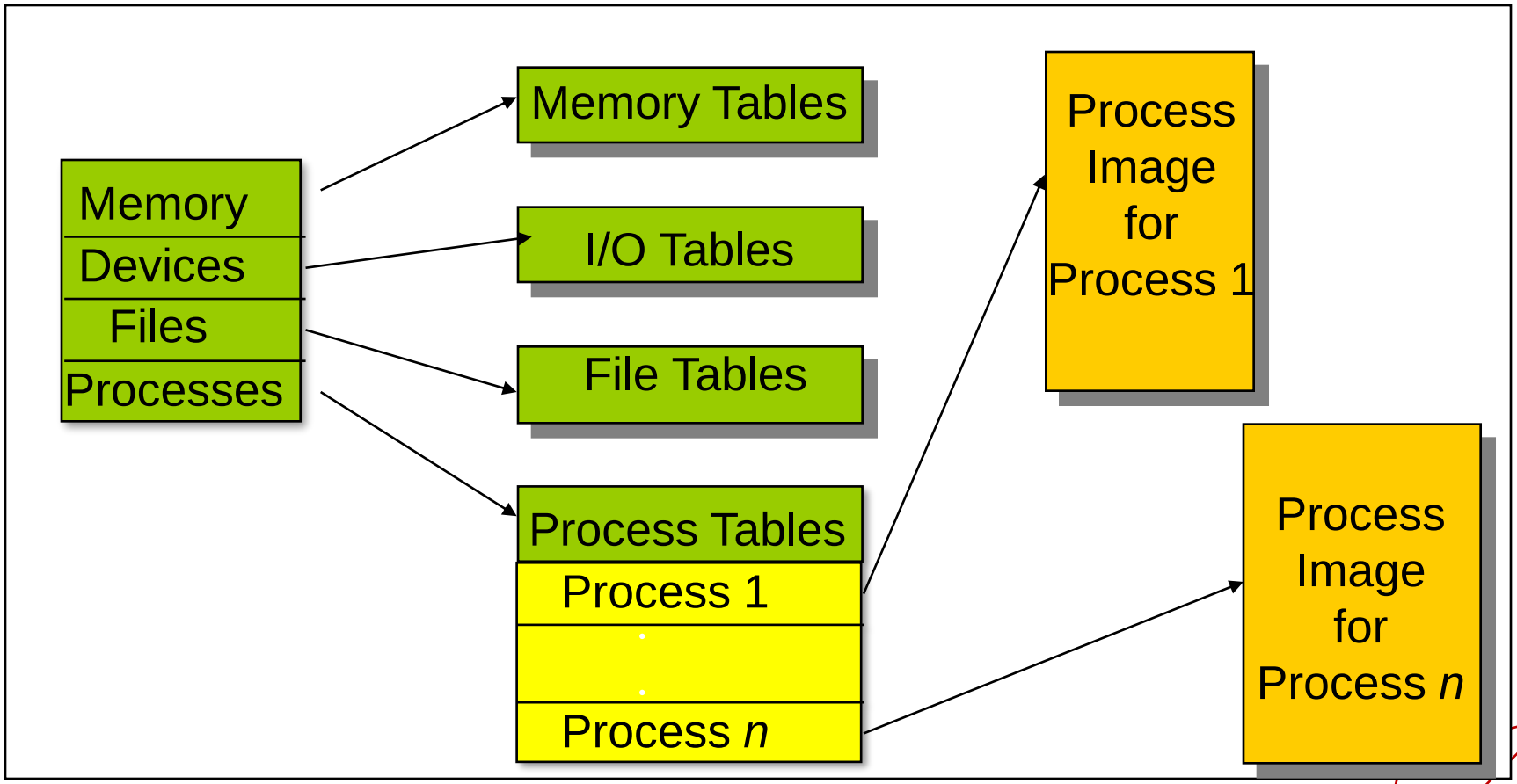


Information Table (cont…)

- OS needs to **keep track** on the **status** of each **process** as well as the resources available in a computer system.
- Done by keeping the information in **tables** as follows:



Information Table (…cont)



3.1.3 Process Control Block (PCB)

Information associated with each process

(also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

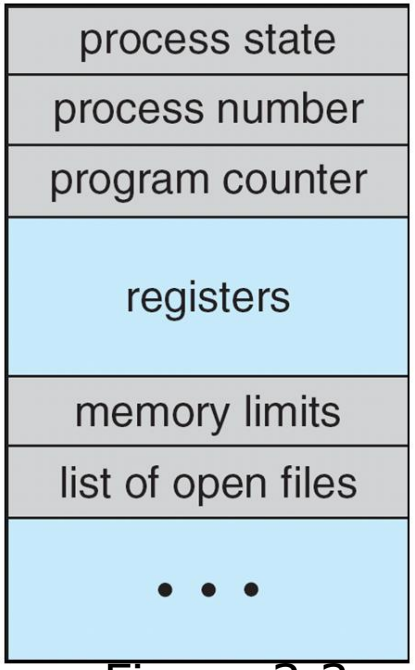
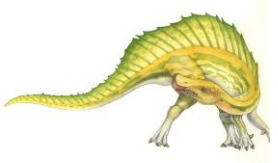
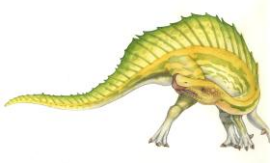


Figure 3.3



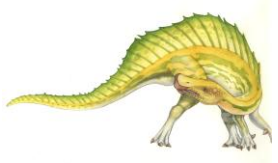
3.1.4 Threads

- So far, process has a single thread of execution
- Consider having multiple program counters per process
  - Multiple locations can execute at once
    - Multiple threads of control -> **threads**
- Need storage for thread details, multiple program counters in PCB
- Covered in the next chapter



3.2 Process Scheduling

- Maximize CPU use
  - Quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device (Figure 3.5)
  - Processes migrate among the various queues
- A common representation of process scheduling is a queueing diagram. (Figure 3.6 in slides #11)



Ready Queue And Various I/O Device Queues

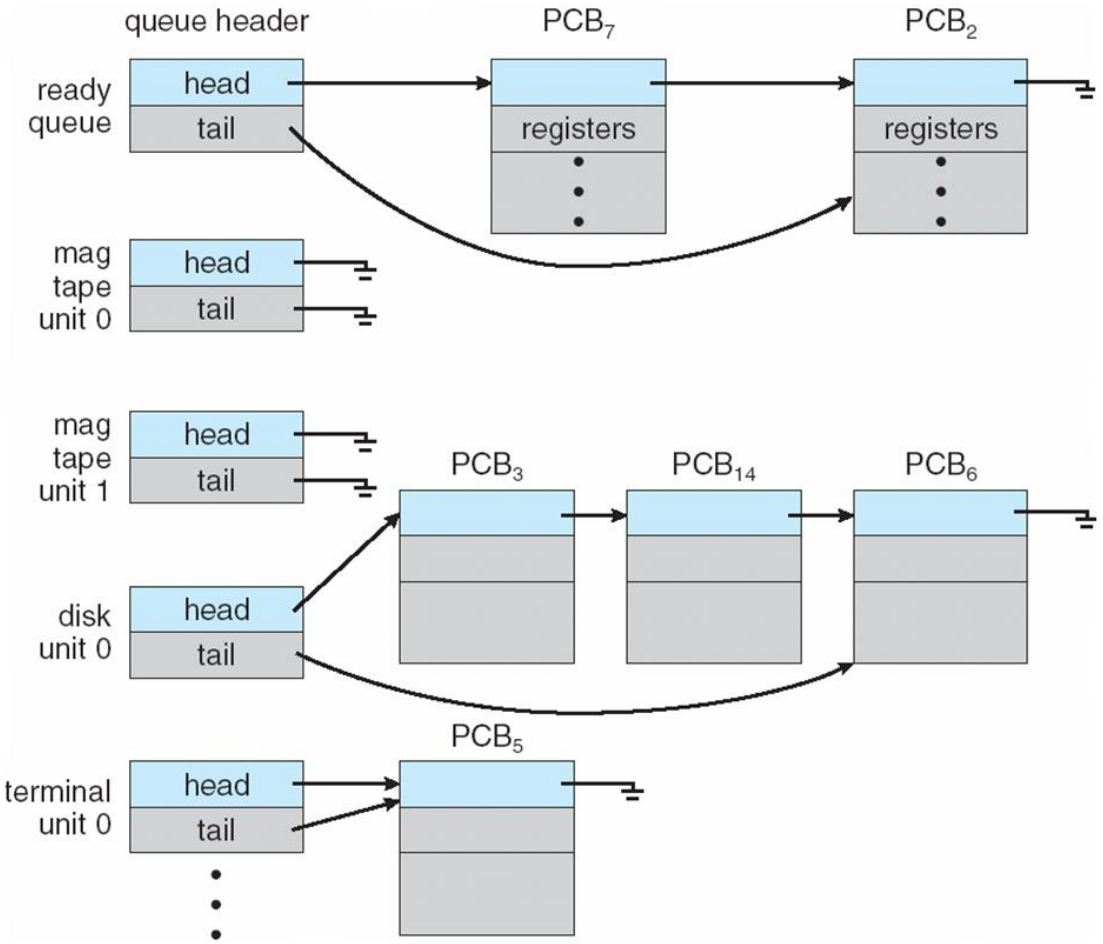
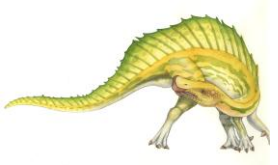
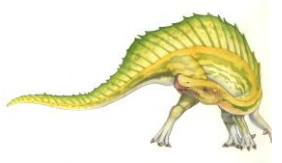


Figure 3.5



3.2.2 Schedulers

- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates a CPU
  - Sometimes the only scheduler in a system
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***



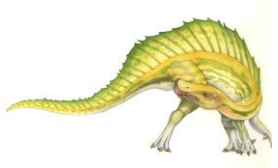
. . . Process Scheduler (cont'd)

- **Middle-level scheduler:** third layer
- Found in highly interactive environments
  - Handles overloading
    - Removes active jobs from memory
    - Reduces degree of multiprogramming
  - Results in faster job completion

- Single-user environment
  - No distinction between job and process scheduling
  - One job active at a time
    - Receives dedicated system resources for job duration

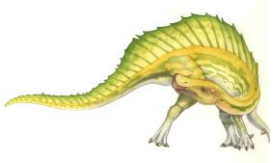
Multitasking in Mobile Systems

- Some mobile systems (e.g., early version of iOS) allow only one process to run, others suspended
- Starting with iOS 4, it provides for a
  - Single **foreground** process – controlled via user interface
  - Multiple **background** processes – in memory, running, but not on the display, and with limits
  - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback
- Android runs foreground and background, with fewer limits
  - Background process uses a **service** to perform tasks
  - Service can keep running even if background process is suspended
  - Service has no user interface, small memory use



3.2.3 Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is pure overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once



CPU Switch From Process to Process

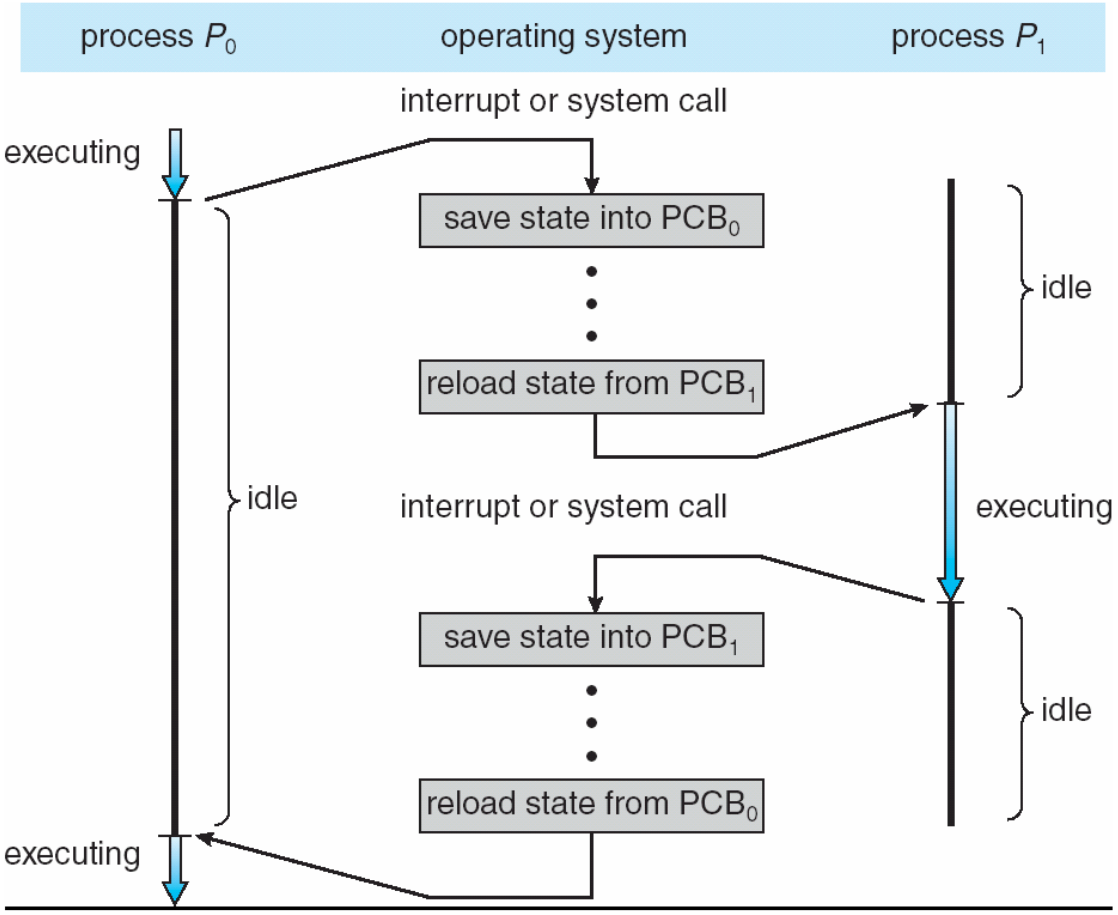


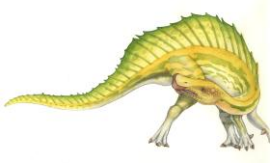
Figure 3.4

When to Switch a Process

- **Clock interrupt**
  - process has executed for the maximum allowable time slice
- **I/O interrupt**
- **Memory fault**
  - memory address is in virtual memory so it must be brought into main memory
- **Trap**
  - error occurred
  - may cause process to be moved to Exit state
- **Supervisor call**
  - such as file open

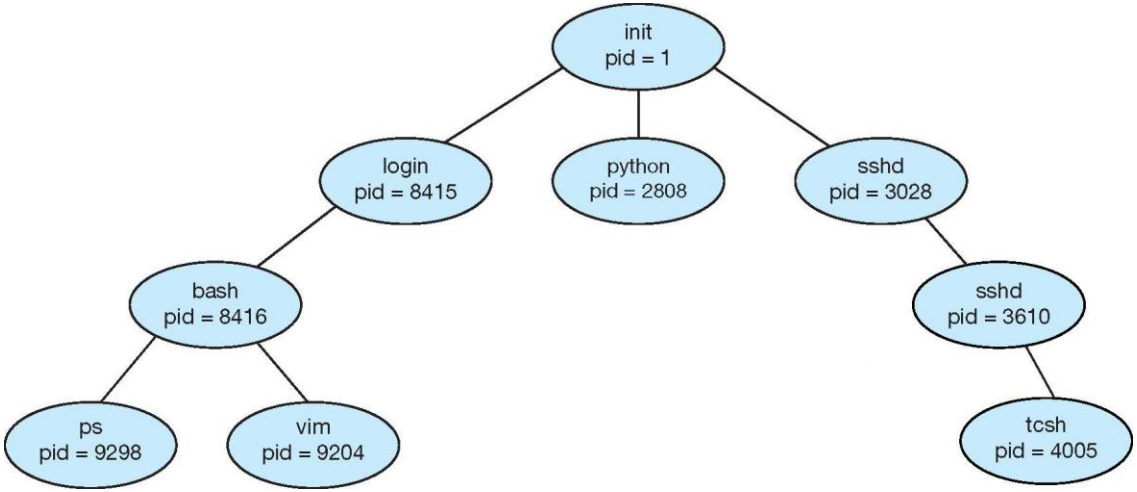
3.3 Operations on Processes

- System must provide mechanisms for:
  - process creation,
  - process termination,
  - and so on as detailed next



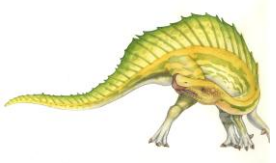
3.3.1 Process Creation

- A **process** may create other processes.
- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, a process is identified and managed via a **process identifier** (**pid**)
- A Tree of Processes in UNIX



Process Creation (Cont.)

- Resource sharing among parents and children options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate



Process Creation (Cont.)

- Address space
  - A child is a duplicate of the parent address space.
  - A child loads a program into the address space.
- UNIX examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** replaces the process' memory space with a new program

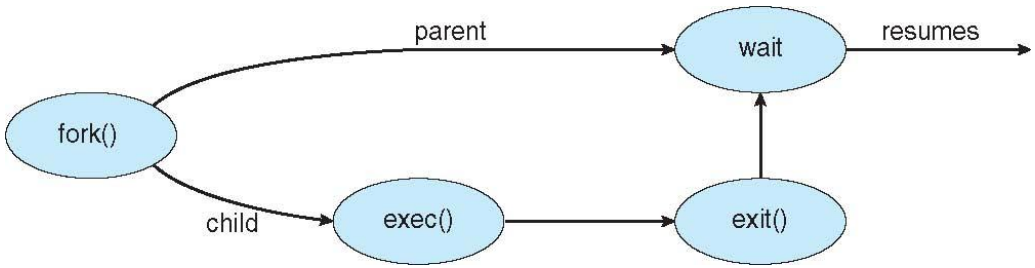


Figure 3.10 Process creation using the fork() system call

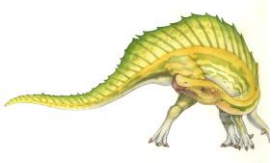
C program to create a separate process in UNIX

```
int main()
{
pit.t pid;
      /*fork a child process */
      pid = fork();

      if (pid < 0) { /* error occurred */
      fprintf(stderr, "Fork Failed");
      return 1;
      }
      else if (pid == 0) { /*child process */
         execlp("/bin/ls","ls",NULL);
      }
      else { /* parent process */
         /* parent will wait for the child to complete */
         wait(NULL);
         printf("Child Complete");
      }
      return 0;
}
```

Figure 3.9

C Program Forking Separate Process

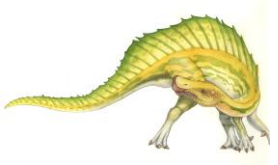
- The C program illustrates how to create a new process UNIX.
- After **`fork()`** there are two different processes running copies of the same program. The only difference is that the value of pid for the child process is zero, while that for the parent is an integer value greater than zero
- The child process inherits privileges and scheduling attributes from the parent, as well certain resources, such as open files.
- The child process then overlays its address space with the UNIX command "ls" (used to get a directory listing) using the execlp() (a version of the exec() system call).
- The parent waits for the child process to complete with the wait() system call.
- When the child process completes the parent process resumes from the call to wait(), where it completes using the exit() system call.



Creating a Separate Process via Windows API

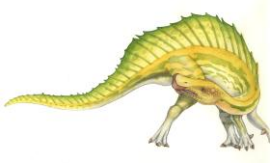
```
int main(VOID)
{
STARTUPINFO si;
PROCESS_INFORMATION pi;

     /* allocate memory */
     ZeroMemory(&si, sizeof(si));
     si.cb = sizeof(si);
     ZeroMemory(&pi, sizeof(pi));

     /* create child process */
     if (!CreateProcess(NULL, /* use command line */
     "C:\\WINDOWS\\system32\\mspaint.exe",/* command
*/
     NULL, /* don't inherit process handle */
     NULL, /* don't inherit threat handle */
     FALSE, /* disable handle inheritance */
     0, /* no creation flags */
     NULL, /* use parent's environment block */
     NULL, /* use parent's existing directory */
     &si,
     &pi))
     {
     {
     fprintf{stderr, "Create
Process Failed"};
     return -1;
     }
     /* parent will wait for the
child to complete */

WaitForSingleObject(pi.hProcess,
INFINITE);
     printf("Child Complete");

     /* close handles */
     CloseHandle(pi.hProcess);
     CloseHandle(pi.hThread);
}
```

Figure 3.11



3.3.2 Process Termination

- A process terminates when it finishes executing its final statement and it asks the operating system to delete it by using the **`exit()`** system call.
  - At that point, the process may return a status value (typically an integer) to its parent process (via the wait() system call.
  - All the resources of the process are deallocated by the operating system.
- A parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates



Process Termination (Cont.)

- Some operating systems do not allow a child process to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc. are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the `wait()` system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

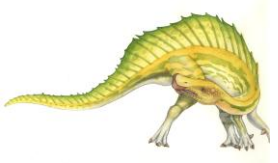
- If no parent waiting (did not invoke `wait()`) process is **zombie**
- If parent terminated without invoking `wait`, process is **orphan**

Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - **Browser** process manages user interface, disk and network I/O
  - **Renderer** process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
    - Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
  - **Plug-in** process for each type of plug-in

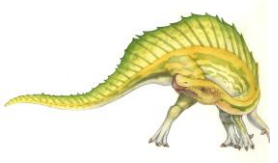
3.4 Interprocess Communication

- Processes within a system may be ***independent*** or ***cooperating***
  - Cooperating processes can affect or be affected by other processes, including sharing data
  - Independent processes cannot affect other processes
- Reasons for having cooperating processes:
  - Information sharing
  - Computation speedup (multiple processes running in parallel)
  - Modularity
  - Convenience

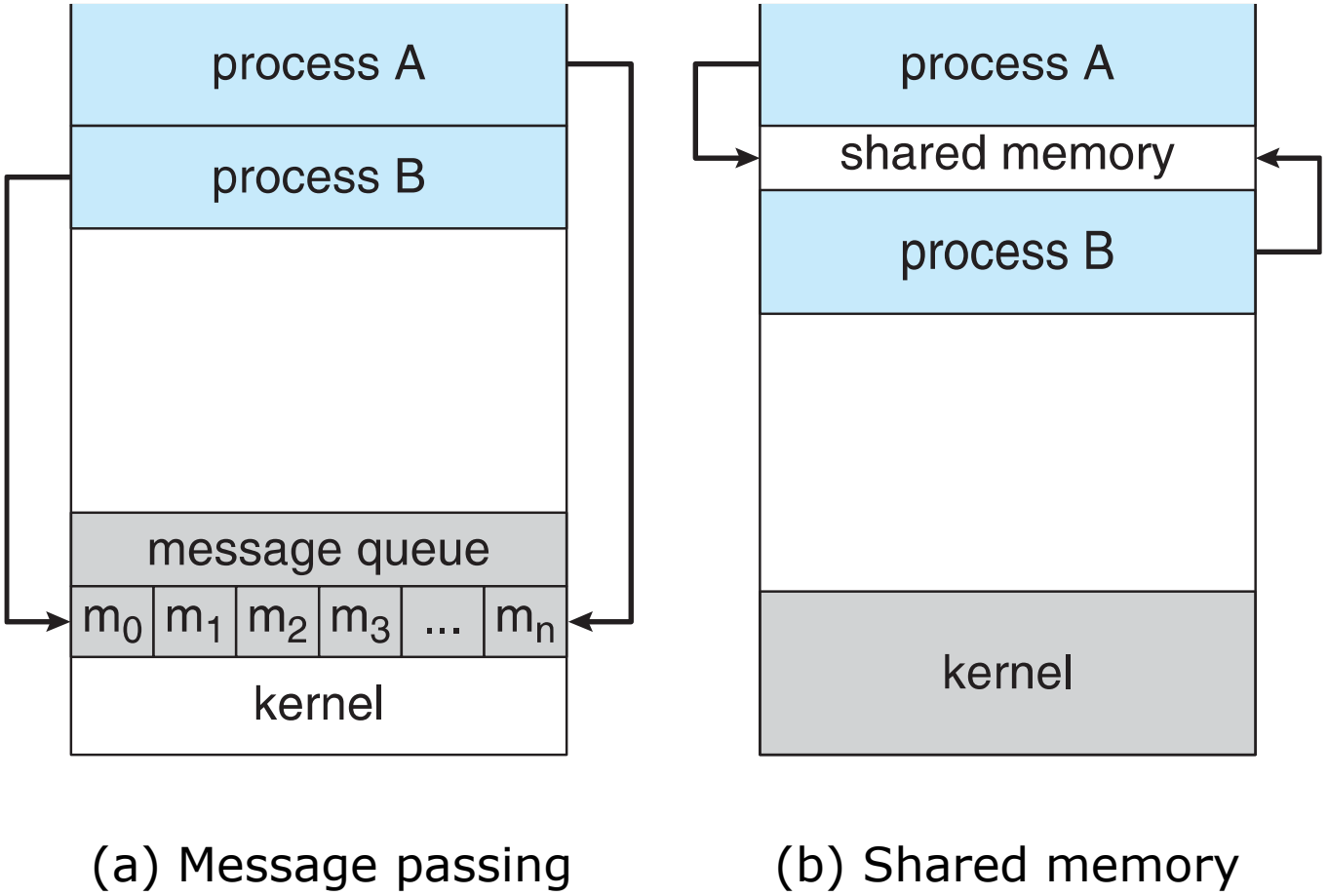


Interprocess Communication

- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

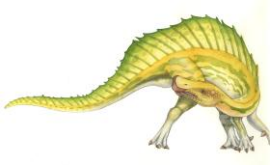


Two Models of IPC



(a) Message passing

(b) Shared memory



Shared Memory: Producer-Consumer Problem

- Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process
  - **unbounded-buffer** places no practical limit on the size of the buffer
  - **bounded-buffer** assumes that there is a fixed buffer size
- Will be detailed out in Chapter 6 – Synchronization.

Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10

typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];

int in = 0;

int out = 0;
```

Details in Chap. 6 !

- Solution is correct, but can only use BUFFER_SIZE-1 elements

Bounded-Buffer – Producer

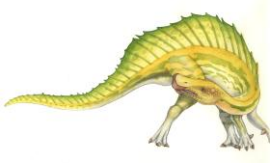
Details in Chap. 6 !

```
item next produced;
while (true) {
   /* produce an item in next produced */
   while (((in + 1) % BUFFER SIZE) == out)
       ; /* do nothing */
   buffer[in] = next produced;
   in = (in + 1) % BUFFER SIZE;
}
```

Bounded Buffer – Consumer

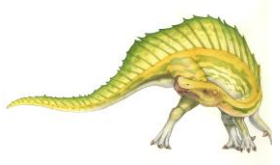
```
item next consumed;

while (true) {
        while (in == out)
                ; /* do nothing */
        next consumed = buffer[out];

        out = (out + 1) % BUFFER SIZE;

        /* consume the item in next consumed */

}
```

Details in Chap. 6 !



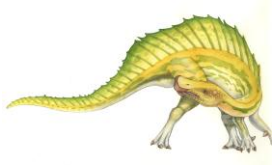
IPC – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **`send`**(*message*) – message size fixed or variable
  - **`receive`**(*message*)



IPC – Message Passing

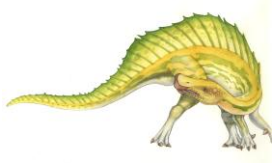
- If *P* and *Q* wish to communicate, they need to:
  - establish a ***communication link*** between them
  - exchange messages via send/receive
- Implementation of communication link
  - physical (e.g., shared memory, hardware bus)
  - logical (e.g., direct or indirect, synchronous or asynchronous, automatic or explicit buffering)



Direct Communication

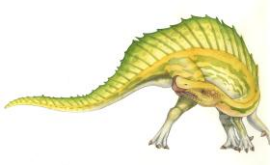
- Processes must name each other explicitly:
  - **`send`** (*P, message*) – send a message to process P
  - **`receive`**(*Q, message*) – receive a message from process Q

- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional



Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional



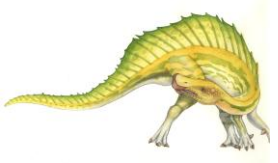
Indirect Communication

- Operations
  - create a new mailbox
  - send and receive messages through mailbox
  - destroy a mailbox

- Primitives are defined as:

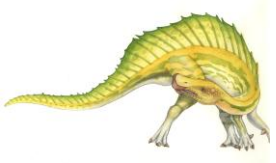
  `send`(*A, message*) – send a message to mailbox A

  `receive`(*A, message*) – receive a message from mailbox A



Indirect Communication

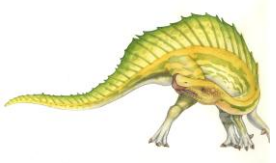
- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.



Synchronization

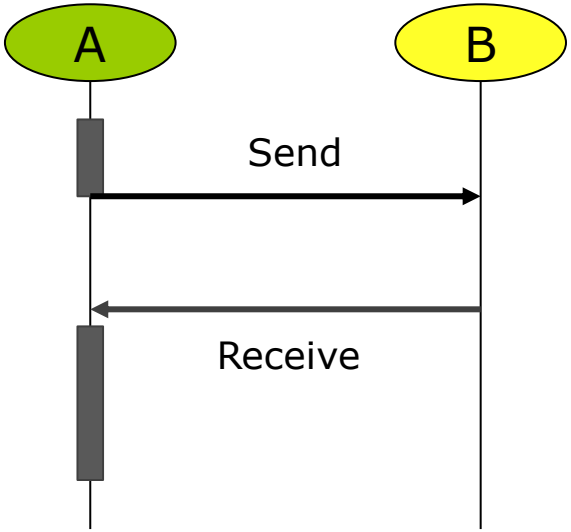
- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
  - **Blocking send** has the sender block until the message is received
  - **Blocking receive** has the receiver block until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking** send has the sender send the message and continue
  - **Non-blocking** receive has the receiver receive a valid message or null

}

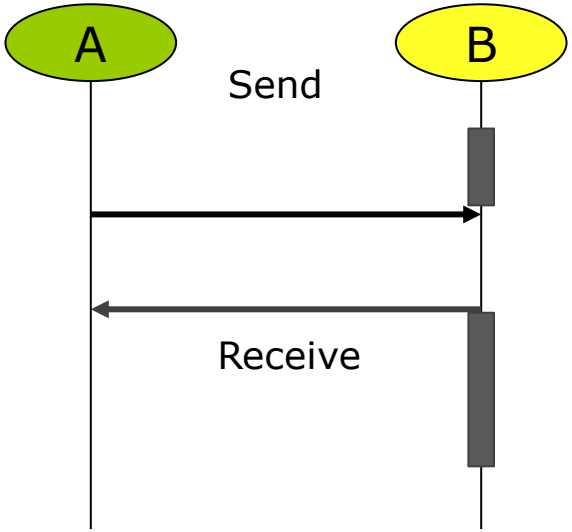


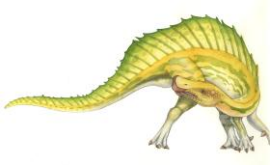
Synchronization

Blocking Send



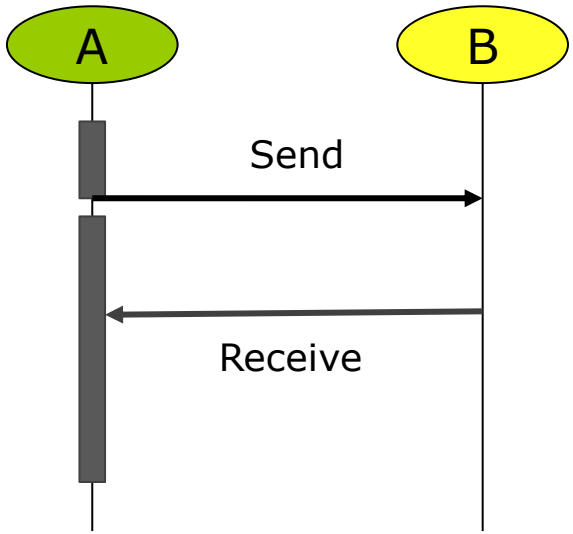
Blocking Receive



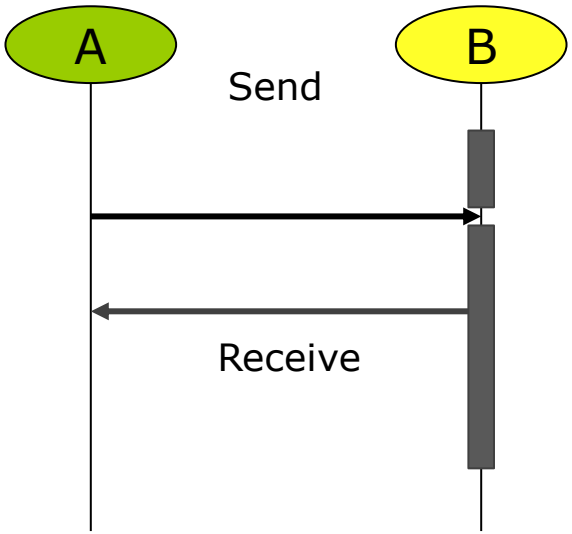


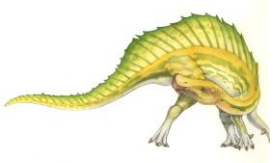
Synchronization

Non-Blocking Send



Non-Blocking Receive



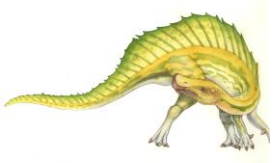


EXERCISE

From Silberchatz, Operating System Concepts Chapter 3 Exercises

- Odd number Matric card – 3.1, 3.3, 3.5
- Even number Matric card – 3.2, 3.4, 3.9

- Due date: Submit to e-learning 5 days after the end of the lecture.



End of Chapter 3